UNIVERSIDAD DEL VALLE — FACULTAD DE INGENIERÍA

Fundamentos de Programación Orientada a Objetos (FPOO)

INFORME DE PRIMERA ENTREGA PROYECTO: BUS JAM (C++)

———————————————————————————————————————

Estudiante(s):	Samuel Martínez Muñoz
	Sofía Montealegre Polanco
	Samuel Tellez Caicedo
	Alejandro Murillo Renteria
	Jorge Andres Rodriguez Beltran
Código(s):	202535015
	202129809
	202540928
	202538979
	202618237
Correo Institucional:	samuel.martinez.munoz@correounivalle.edu.co
	sofia.montealegre@correounivalle.edu.co
	samuel.tellez@correounivalle.edu.co
	alejandro.murillo@correounivalle.edu.co
	jorge.rodriguez.beltran@correounivalle.edu.co
Fecha de Entrega:	Noviembre 2026

1. Requerimientos Identificados e Historias de Usuario

A continuación se detallan los requerimientos funcionales del sistema estructurados bajo el formato de Historias de Usuario (HU), sus criterios de aceptación y las tareas técnicas de implementación en C++:

ID	Historia de Usuario	Criterios de Aceptación	Tareas de Implementación
HU-01	Carga de Archivos de Nivel: Como jugador, quiero seleccionar un nivel desde el menú para que la cuadrícula, los vehículos y los pasajeros se instalen dinámicamente desde un archivo plano.	1. Apertura y validación del archivo .txt en rutas relativas. 2. Carga precisa de dimensiones N x M, vehículos (posición, tipo, orientación, color, capacidad) y cola de pasajeros.	- Creación del método JuegoModel::cargarNivel(). - Parsing de cadenas de texto a enumeraciones Color y Orientacion. - Instanciación dinámica de Carro, Bus y Pasajero.
HU-02	Movimiento de Vehículos: Como jugador, deseo ingresar el ID de un vehículo para desplazarse en su eje hasta la Zona de Parqueo.	1. El vehículo solo se desplaza si su trayectoria hasta el borde está 100% libre de colisiones. 2. Si la Zona de Parqueo tiene espacio (< 4), el vehículo sale de la matriz y se estaciona.	- Implementación de Tablero::caminoDespejadoHastaSalida(). - Implementación de Tablero::hayVehiculoEn(). - Coordinación desde JuegoController::seleccionarYMoverVehiculo().
HU-03	Gestión de Filas y Parqueo: Como jugador, quiero que los vehículos estacionados en parqueo reciba pasajeros de su mismo color.	1. Si el pasajero en embarque coincide en color con un bus estacionado, aborda automáticamente. 2. Si no coincide, la fila circular rota para evaluar el siguiente pasajero.	- Implementación de la FilaEstatica (FIFO) y FilaRotatoria (circular). - Métodos de ZonaParqueo para controlar capacidad. - Definición del ciclo JuegoModel::ejecutarCicloAbordaje().

2. Documentación de Tarjetas CRC (Class Responsibility Collaborator)

Las tarjetas CRC resumen las responsabilidades principales de cada clase dentro del patrón MVC y sus interacciones/colaboraciones del dominio del sistema:

Clase Abstracta: Vehiculo	*Capa: Modelo*
Responsabilidades: - Servir de clase base abstracta para entidades móviles. - Almacenar posición (X,Y), orientación, color, tamaño y pasajeros actual/máximo. - Controlar abordaje y estado de capacidad completa (estaLleno).	**Colaboradores:** - Enums (Orientacion, Color)

Clases Concretas: Carro / Bus	*Capa: Modelo*
Responsabilidades: - Heredar de Vehiculo e implementar la firma virtual getType(). - Carro: Define largo 2 y capacidad fija de 2 pasajeros. - Bus: Define dimensiones y capacidad parametrizables según el nivel.	**Colaboradores:** - Vehiculo (Clase Padre)

Clase: Tablero	*Capa: Modelo*
Responsabilidades: - Gestionar la matriz del terreno (N x M) y la lista de vehículos activos. - Buscar vehículos por ID y verificar colisiones en celdas. - Validar el camino libre en el eje del vehículo hacia la salida.	**Colaboradores:** - Vehiculo

Clases: FilaEstatica / FilaRotatoria	*Capa: Modelo*
Responsabilidades: - FilaEstatica: Mantener la cola FIFO inicial de pasajeros. - FilaRotatoria: Administrar el arreglo circular de	**Colaboradores:** - Pasajero

embarque (capacidad 4) y ejecutar la rotación de posiciones.	

Clase: ZonaParqueo	*Capa: Modelo*
Responsabilidades: - Administrar las 4 posiciones de estacionamiento para vehículos listos para embarcar. - Recibir vehículos procedentes del tablero y remover los que completen su cupo.	**Colaboradores:** - Vehiculo

Clase: JuegoModel	*Capa: Modelo*
Responsabilidades: - Cargar y parsear la información de niveles desde archivos planos (.txt). - Mantener el estado global del juego (tablero, filas, parqueo, victoria/derrota).	**Colaboradores:** - Tablero, FilaEstatica, FilaRotatoria, ZonaParqueo

Clases: JuegoView / JuegoController	*Capa: Vista / Controlador*
Responsabilidades: - JuegoView: Renderizar menús y estado del tablero/filas en consola. - JuegoController: Capturar comandos de entrada del usuario y solicitar la lógica de movimiento al modelo.	**Colaboradores:** - JuegoModel

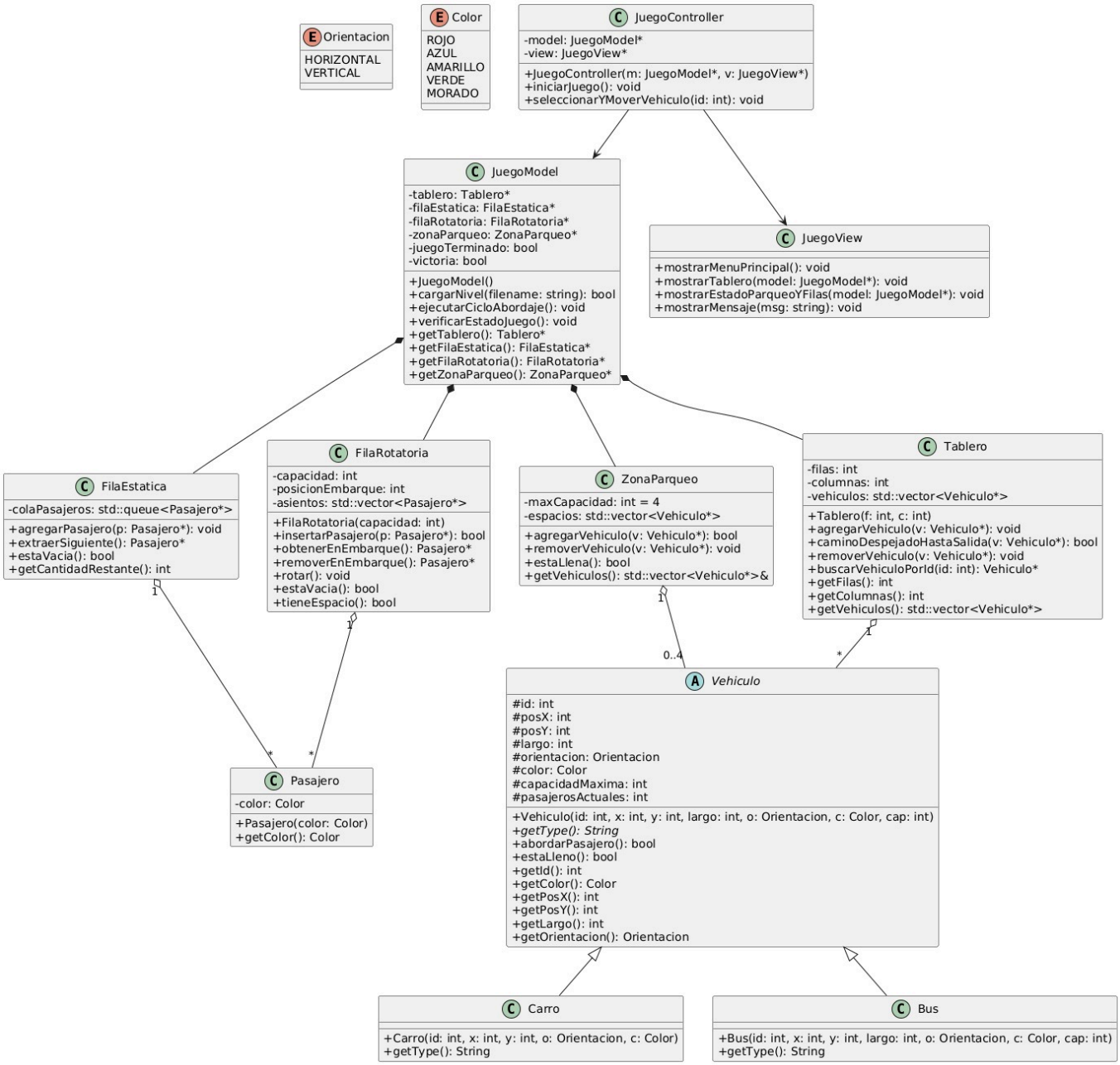
3. Arquitectura del Sistema y Diagrama UML

El diseño arquitectónico del proyecto está fundamentado en el patrón Modelo-Vista-Controlador (MVC), garantizando un alto desacoplamiento y cumpliendo con los principios SOLID:

• Modelo (JuegoModel, Tablero, Vehiculo, Pasajero, Filas, Parqueo): Representa el estado interno del juego, las entidades y las reglas de negocio.
• Vista (JuegoView): Encargada de renderizar por consola la cuadrícula del tablero y las opciones del menú principal.

• Controlador (JuegoController): Captura los comandos de usuario, interactúa con la vista e invoca las mutaciones necesarias en el modelo.

Además, la relación entre Vehículo, Carro y Bus aplica Polimorfismo mediante el método virtual puro getType(), permitiendo extender nuevos tipos de transportes de forma limpia.



4. Notas Adicionales y Observaciones del Grupo

1. La carpeta inicial PROYECTO, tiene otra carpeta adentra llamada PROYECTO, en caso de querer compilar y ejecutarse y no poder, se debe ingresar a la carpeta del interior por medio del comando CD PROYECTO.

2. Se ejecuta el código con la siguiente información en pantalla: Cargar nivel, que luego dirige al usuario a 3 opciones, los cuales son los niveles que el usuario tiene para escoger, una vez el nivel se ha elegido, el programa se dirige al tablero de juego, donde se mostrará información sobre vehículos restantes en cuadrícula, vehículos en parqueo, pasajeros restantes, ingresar el ID del vehículo que se desea mover, y ademas la opcion de salir hacia el menú. En consecuencia, también se encuentra la opción de salir del juego, el cual termina la ejecución del programa.